与 AI Agent 协同的护眼工作流程

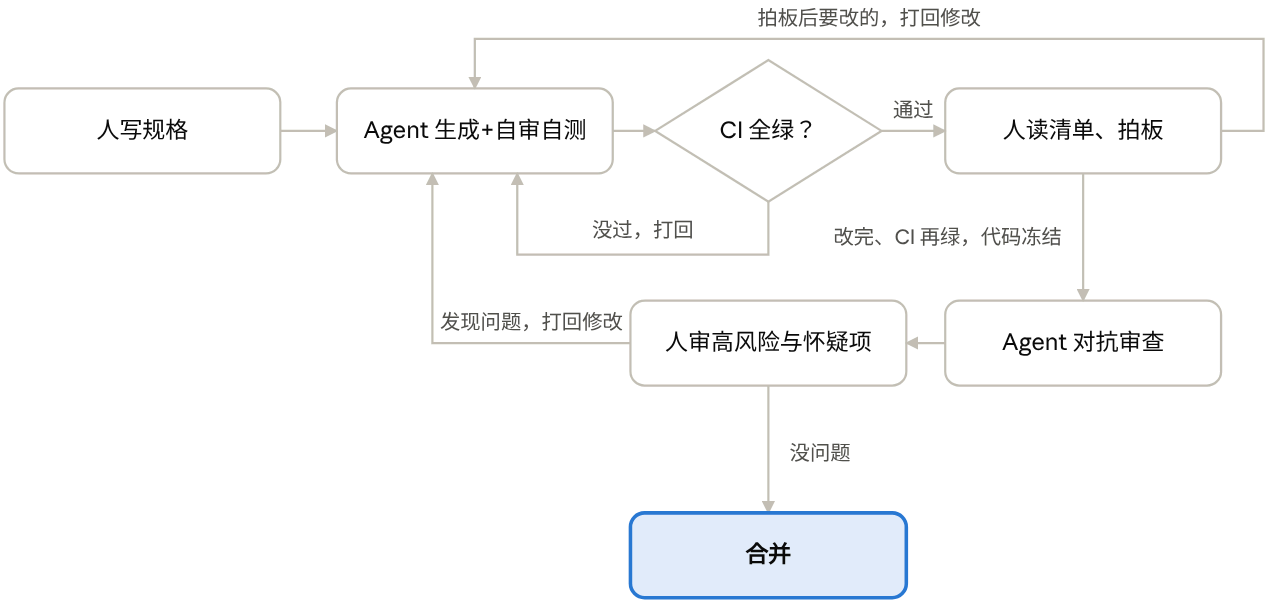
核心原则

眼睛累的根源是“看屏幕的总量”，所以这套流程的目标是减少需要人眼看的内容，而不是让人看得更快。每一条规则都围绕四件事：

1. 把人的工作前移到代码生成之前（写规格、审方案），那是字数最少、杠杆最大的阶段。
2. Agent 运行时不盯屏，这段时间用来做不费眼的事。
3. 审查分层，机器和 Agent 先过滤，人只看高风险和它们过不了的部分。
4. 生成速度受限于审查速度，审不过来就限流，不硬审。

一个工作循环

人只做三件事：写规格、拍板清单、审高风险



一个任务的工作循环 · 7 步，3 个打回点

一个任务从规格到合并走七步，人出现在三处：写规格、读清单拍板、审高风险，其余交给 Agent 和 CI。CI 没过的直接打回 Agent，不占用人的眼睛；拍板后要改的先改完再冻结代码，对抗审只审冻结后的版本；人审发现的问题也打回 Agent 修，人不动手改代码。

关键是错开：Agent 生成任务 A 的时候，人在写任务 B 的规格或审任务 A 的上一个产出。人和 Agent 都不空转，但人看屏幕的时间只集中在写规格和审高风险这两段。

单个任务里你要做的事，按顺序勾着走：

- [ ] 1 写规格：目标、范围、验收标准、禁区，并写明交付前要自审、要附人工审查清单（用“启动前”里的模板）
- [ ] 2 看方案：Agent 给出改哪些文件、怎么改、风险，几百字，批准或改完再放行
- [ ] 3 启动生成，离开屏幕：去写下一个规格或审上一个任务
- [ ] 4 Agent 在同一会话自审并列出人工审查清单：对照验收标准逐条核对、跑全部测试、重读 diff；再列出需要人拍板的决定和它拿不准的代码。这步你不参与
- [ ] 5 收到完成通知，只看两个信号：CI 是否全绿、自审记录是否逐条对上验收标准。缺一个直接打回，什么都不读
- [ ] 6 读人工审查清单：“需要人拍板”的项当场定下来，连同要改的地方一次性回复同一会话让它修；“拿不准”的项记进自己的待看清单
- [ ] 7 修改完、CI 再次全绿后，代码冻结：启动对抗审查会话，离开屏幕等报告。冻结期间不让写代码的会话再动代码
- [ ] 8 读对抗审报告：“确认”项记下来，“怀疑”项记进待看清单。先不打回，攒到第 11 步一起
- [ ] 9 看测试：断言对不对，验收标准是否都覆盖了
- [ ] 10 逐行看两类代码：风险表里的高风险区域，以及待看清单上的项
- [ ] 11 把第 8 到 10 步发现的问题汇总，一次性打回 Agent 修；修完让对抗审会话只看增量 diff 确认修复，不整体重跑
- [ ] 12 合并，勾掉规格里的验收标准，开始下一个任务

第 3、4、7 步是不看屏幕的时间，第 9、10 步是唯一需要逐行看代码的时间。一条硬规则贯穿第 7 步之后：审查期间代码冻结，发现的问题攒着一次打回，不要边审边改。边审边改代码就在动，对抗审看的就是旧版本，报告出来已经对不上了。

人工审查清单、自审、对抗审各管一段：清单是 Agent 自己知道拿不准的地方，自审抓漏项和对不上验收标准的地方，这两样都和写代码时有同样的盲区；对抗审换一个会话，专门抓它自己看不见的问题。所以风险表里的高风险区域不管清单有没有提都要人看，三道关缺一不可。

启动前：写规格

规格写得越清楚，后面要看的代码越少，这是整个流程里最省眼睛的一步。规格是给 Agent 的，也是给审查时的自己的：审查就是对照规格验收。

写规格的三条规则：

1. 一个任务只做一件事。混着重构的大任务会产出大 diff，审查成本成倍上升。宁可拆成三个小任务。
2. 先要方案，再要代码。让 Agent 先用几百字说清楚打算改哪些文件、怎么改、有什么风险，你批了它再动手。方向性错误在这一步截住，代价是读几百字而不是几千行。
3. 写下验收标准和禁区。验收标准是审查时唯一要核对的东西；禁区（不许动的文件、不许改的格式、不许顺手重构）能大幅减少 diff 噪音。

可以直接套用的规格模板：

```
## 任务
一句话说清楚要达成什么。

## 范围
- 涉及的模块/文件：
- 不许动的部分：

## 验收标准
- [ ] 条件 1（可验证的行为，而不是“正常工作”）
- [ ] 条件 2
- [ ] 已有测试全部通过，新增逻辑有测试覆盖

## 约束
- 不做无关重构，不改格式
- 一个 PR 只做这一件事
- 先给出方案（改哪些文件、怎么改、风险），等确认后再写代码

## 交付前自审（在本会话完成，结果写进自述）
- 对照验收标准逐条核对，每条标注通过/未通过及依据
- 运行全部测试和检查，附上结果
- 重读一遍 diff，找出自己不确定的地方

## 人工审查清单（自审完成后单独列出）
列出需要人工审查的项目，分两类：
1. 需要人拍板的决定：业务规则的取舍、接口和数据结构的设计、你不该替人决定的事
2. 你拿不准的代码：不确定对不对、测试没覆盖到、依赖了未验证假设的地方
每项写清楚：位置、为什么需要人看、你倾向的做法和理由、如果判断错了会怎样。
按后果严重程度排序，最多 10 条，宁缺毋滥。没有就明确说没有。

## 交付时附带
```

```
- 改了什么、为什么
- 自审结果：每条验收标准的核对情况
- 人工审查清单
- 哪些你没有验证
```

写规格不需要盯屏幕：可以口述给语音输入，或者在纸上先列要点再誊进去。

Agent 运行中

Agent 跑的时候不看它的输出，这段时间是你一天里最该离开屏幕的时候。它跑完之前你什么都改不了，盯着看只消耗眼睛。

该做的事，按优先级：

1. 写下一个任务的规格（纸上或语音都行）。
2. 审上一个任务的产出（只看后面“分层审查”里规定要人看的部分）。
3. 做不需要屏幕的思考：架构取舍、需求里模糊的点、要跟谁确认什么。
4. 什么都不做。站起来、看远处、喝水。判断力和眼睛都需要休息。

不该做的事：

- 盯着 Agent 逐行输出。
- 同时跑超过两个 Agent。两个是舒服的上限：一个在跑、一个在审。再多审查会堆积，又回到看不完的状态。
- 刷无关的东西。碎片化切换之后回来接 Agent 的输出，上下文全丢，得重新读一遍，等于多看一次。

让它跑完后主动通知你（终端提示音、桌面通知、消息推送），而不是你反复切回去查看进度。

分层审查

代码到人眼前要先过三道关，每一道都把一部分问题挡在人工审查之外。人只看最后剩下的高风险部分和怀疑项。

层	谁	看什么	没过怎么办
1 机器	CI	类型检查、lint、格式化、全部测试	直接打回 Agent，人不看

层	谁	看什么	没过怎么办
2 Agent 自审	写代码的 Agent，同一会话	对照验收标准逐条核对、跑全部测试、重读 diff，把结果写成自述，并列出人工审查清单	自审记录缺失或核对不上的直接打回，人不看代码
3 Agent 对抗审	另一个会话	边界、并发、错误处理、安全、与需求的偏差	它报“确认”的先让写代码的 Agent 复现修复；报“怀疑”的进入人审清单
4 人审	你	高风险区域逐行；其余只看自述、测试和怀疑项	打回 Agent 修，人不动手改

人审的顺序，省眼睛的看法：

1. 先读人工审查清单：拍板的回复 Agent，要改的一次打回；改完、CI 再绿后代码冻结。
2. 冻结后启动对抗审，等报告的时间离开屏幕。回来读自述和对抗审报告，决定要不要往下看。
3. 再看测试：断言对不对、覆盖了验收标准没有。测试写得对，实现大概率可以信任。
4. 最后只对高风险区域和待看清单逐行看，问题攒着一次打回。

按风险分级，决定哪些代码值得人眼逐行看：

风险	范围	人怎么看
高	鉴权、支付、数据迁移、并发、对外接口、删除数据的逻辑	逐行看，不省
中	核心业务逻辑、数据模型变更	看测试和接口，实现抽查
低	内部工具、样式、测试代码、文档、配置	只看自述，CI 绿了就合

两条硬规则：没有自述和测试的 PR 不进入人审；审不过来的 PR 不合并，而是减少新任务的启动，直到积压清掉。

对抗审查提示词

骨架固定，每次只换两处：任务背景，和“本次额外重点”里的攻击角度。角色、规则、输出格式不动，这样报告的样子每次一样，读起来最快。

通用骨架：

```
你是一名对抗性代码审查员。假设这段代码一定有问题，把问题找出来。不评价优点，不讨论风格和命名，只关注会导致错误行为、数据损坏、安全漏洞或线上事故的问题。

## 背景
语言/框架：{{语言、框架、运行环境}}
任务目的：{{规格里的“任务”和“验收标准”原文}}
相关约束：{{多实例部署、高并发、处理用户输入、涉及资金等；没有写“无”}}

## 输入
### diff
{{粘贴 diff，附被改动函数的完整上下文}}

### 写代码会话的自述与自审记录
{{粘贴}}

### 它列出的人工审查清单
{{粘贴}}

对这些输入的态度：
- 自述里每一条“已验证”“已处理”，你都要在代码里自己核实，核实不了的按未验证处理。
- 自述没提到、自审记录里一笔带过的地方，是它最自信的地方，也是你的重点。
- 人工审查清单里“拿不准”的每一项，你必须给出明确判断：有问题/没问题，以及依据。

## 攻击角度
逐段读代码，对每一段问：什么输入、什么时序、什么环境会让这里出错？
1. 边界与输入：空、零、负数、极大值、超长、非 ASCII、重复、乱序、类型不符
2. 错误处理：异常被吞、部分失败后状态不一致、重试导致重复副作用、异常路径上资源未释放
3. 并发与时序：竞态、检查-再操作不原子、共享可变状态、多实例下的本地假设
4. 安全：注入、鉴权缺失或可绕过、敏感信息进日志、不安全反序列化、SSRF
5. 数据一致性：事务边界、幂等、迁移不可回滚、时区/精度/编码
6. 隐含假设：代码默默依赖了哪些未验证的前提，什么情况下会被打破
7. 与需求的偏差：验收标准是否真的满足，有没有漏掉的分支或顺手改了不该改的

## 本次额外重点
{{按任务类型从下表挑 2 到 4 条粘进来}}

## 输出
按严重程度排序，每条包含：位置（文件:行号或函数名）、严重程度（致命/高/中/低）、一句话问题、触发场景（具体输入或操作序列）、置信度（确认/很可能/怀疑；怀疑必须说明需要看什么才能确认）。
最后两节单独列：
- 对人工审查清单的逐项判断
```

- 无法验证的部分：看不到的依赖、调用方、配置

规则：宁可少报不报假的；不给笼统建议；不修代码，只指问题；没发现中以上问题就直说，并说明检查了哪些路径。

按任务类型换“本次额外重点”：

任务类型	额外攻击角度
新功能、业务逻辑	验收标准逐条对照；漏掉的分支和状态；重复提交是否幂等；旧数据是否兼容
Bug 修复	修的是根因还是症状；同类问题在别处是否还在；有没有复现该 bug 的测试；修复是否影响了相邻行为
重构	行为是否严格等价；被“顺手”删掉的边界处理；公共接口有没有变；性能有没有退化
数据库、迁移	能否回滚；大表加锁和耗时；回填数据的正确性；新旧代码共存期间谁读谁写
对外接口、API	向后兼容；输入校验；鉴权在每个入口都有；错误码和错误信息是否泄露内部细节；限流
鉴权、支付、权限	绕过路径；权限提升；金额精度和舍入；重复扣款；敏感字段进日志或返回体
并发、异步、定时任务	竞态；重试和超时；消息重复、乱序、丢失；部分失败后的补偿；多实例同时跑
前端、UI	空态、加载态、错误态；状态与服务端不同步；XSS；异常输入；无障碍
测试代码	断言是否真的断言了（恒通过的测试）；mock 掩盖了什么；覆盖的是验收标准还是实现细节
配置、基础设施	环境间差异；密钥是否暴露；默认值是否安全；能否回滚

第 11 步修完后的增量确认，换这段，不要整体重跑：

```
这是针对你上一轮报告的修复，只审增量 diff，不重新审整个 PR。
逐条核对你报告的每个问题：
- 已修复 / 未修复 / 修法有问题（绕过、治标、引入新问题），附依据
- 修复的改动范围是否超出了报告的问题，超出的部分列出来
新发现的问题单独列在最后，格式同上。

{{上一轮报告}}
{{增量 diff}}
```

工具与环境设置

工具设置的目标有两个：让需要看的 diff 更少，让必须看的内容更容易看。一次配好，长期省眼睛。

减少要看的内容：

- 格式化和 lint 在提交前自动跑，格式类 diff 永远不出现在审查里。
- 给 Agent 一份项目级的固定约束文件（不动的目录、不改的格式、提交前必须跑的命令），每个任务自动带上，不用每次重写。
- 让 Agent 自己跑测试并把结果附在自述里，人不再自己跑一遍。
- 开启 diff 工具的“忽略空白”和“折叠未改动区域”。

让必须看的内容更舒服：

- 编辑器和终端字号比习惯的大一到两档，行高 1.5 以上。
- 代码审查用并排 diff，不用上下 diff，减少眼睛来回跳。
- 屏幕亮度跟环境光匹配，不要在暗房间里看亮屏；浅色还是深色主题以你眼睛舒服为准。
- 显示器放在一臂距离、屏幕上沿与视线齐平。
- 读长篇自述或方案时，可以用朗读功能听，而不是看。

通知：Agent 完成、CI 结果、对抗审报告，全部推送到你面前，不要让自己定时去看。

用眼规则与一天的节奏

把审查集中到固定时段，而不是全天零散地看，眼睛才有真正的休息；Agent 运行的空档就是天然的休息点。

硬性规则：

- 每看 20 分钟屏幕，看 6 米外的东西 20 秒（20-20-20）。Agent 跑起来的那一刻就是执行的时机。

- 连续审查不超过 50 分钟，之后离开屏幕 10 分钟。
- 刻意眨眼。盯屏幕时眨眼频率会降到平时的三分之一，眼干眼涩多半来自这里。
- 已经出现持续酸胀、干涩、视物模糊，去看一次眼科，不要等。

一天的参考节奏（按自己的作息平移）：

时段	做什么	看屏强度
上午第一段	写当天前两三个任务的规格；启动第一个 Agent	低
上午第二段	Agent 跑，你审前一天积压或写下一个规格	中
午饭后	处理 Agent 问题和打回，不做逐行审	低
下午集中审查时段	只审高风险 PR，连续不超过 50 分钟	高
下午最后一段	启动可以过夜跑的任务；清理明天的规格	低

只安排一个“高强度看屏”时段，其余时间的看屏都是短时的。眼睛最累的逐行审查放在精神最好的时候，不要放在一天的末尾。

每日检查清单

开工前：

- [ ] 今天的任务都拆成了“一件事一个 PR”
- [ ] 前两个任务的规格已写好，含验收标准和禁区
- [ ] 积压待审的 PR 不超过 3 个；超过就不启动新任务

每个任务：

- [ ] Agent 先给方案，批了再写代码
- [ ] CI 全绿、自审记录和自述齐全、对抗审报告在手，才进入人审
- [ ] 人审先看人工审查清单、自述和测试，只对高风险区域和清单项逐行
- [ ] 发现问题打回 Agent 修，自己不动手

一天结束：

- [ ] 今天逐行审查的总时长没有超过 2 小时
- [ ] 没有“先合并、以后再看”的 PR
- [ ] 明天的第一个规格已经写好